Чисельні методи

Лабораторна робота №6

Методи чисельного розв’язку звичайних диференційних рівнянь.

Виконав:

Студент КН-24

Бевзюк Н.С.

2026

Варіант **1**

1. Мета роботи.

Здобути практичні навички розроблення алгоритмів і програм чисельного розв’язання звичайних диференційних рівнянь першого порядку методом Ейлера.

2. Завдання.

1) записати задачу Коші $y' = \frac{3x^2}{x^3+y+1}$;

2) реалізувати метод Ейлера зі сталим кроком на відрізку [1, 2];

3) забезпечити точність 0,01за рахунок вибору достатньо малого сталого кроку;

4) отримати послідовність наближених значень $y(x)$;

5) побудувати графік наближеного розв’язку;

3. Розрахункові формули чисельного методу.

Для розв'язання задачі Коші чисельним методом Ейлера використовується ітераційний підхід, що базується на апроксимації шуканої функції за допомогою її похідної в початковій точці кожного кроку. Оскільки обрано спосіб забезпечення точності з постійним кроком, величина h залишається незмінною протягом усього процесу обчислень на відрізку $[x_0, a]$.

Основна розрахункова формула для визначення наступного значення функції має вигляд $y_{i+1} = y_i + h * f(x_i, y_i)$, де $f(x_i, y_i)$ — значення правої частини диференціального рівняння в поточній точці. З урахуванням умови заданого варіанта, де $f(x, y) = \frac{3x^2}{x^3+y+1}$, ітераційний процес описується виразом $y_{i+1} = y_i + h \frac{3x^2{}_i}{x^3{}_i+y_i+1}$. Аргумент на кожному кроці змінюється за законом $x_{i+1} = x_i + h$. Для реалізації методу використовуються початкові умови $x_0 = 1$ та $y_0 = 0$, а кількість кроків n визначається як $(a - x_0) / h$. Обчислення проводяться послідовно до досягнення правої межі відрізка a = 2, що дозволяє отримати дискретний набір значень, які з заданою точністю описують поведінку інтегральної кривої.

4.Текст програми.

ПОЧАТОК КОДУ

```
import math
import tkinter as tk
from tkinter import ttk, messagebox
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
NavigationToolbar2Tk

# Задача Коші
def f(x, y):
    denom = x**3 + y + 1
    if abs(denom) < 1e-14:
        raise ZeroDivisionError("У формулі виникло ділення на нуль або дуже мале
число.")
    return 3 * x**2 / denom

# Метод Ейлера
def euler_method(x0, y0, b, eps):
    if b <= x0:
        raise ValueError("Кінець відрізка має бути більшим за початок.")
    n = max(1, math.ceil((b - x0) / eps))
    h = (b - x0) / n
    history = []
    x = x0
    y = y0
    for k in range(n + 1):
        history.append((k, x, y, f(x, y)))
        if k == n:
            break
        y = y + h * f(x, y)
        x = x + h
    return history, h

# Створння графіку
def draw_graph(history):
    ax.clear()
    xs = [row[1] for row in history]
    ys = [row[2] for row in history]
    ax.plot(xs, ys, marker="o", linewidth=2, label="Наближений розв’язок методом
Ейлера")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title("Графік наближеного розв’язку")
```

```
    ax.grid(True, linestyle=":")
    ax.legend()
    canvas.draw()

# Заповнення таб
def fill_table(history):
    for item in table.get_children():
        table.delete(item)
    for k, x, y, slope in history:
        table.insert(
            "",
            tk.END,
            values=(
                k,
                f"{x:.6f}",
                f"{y:.6f}",
                f"{slope:.6f}"
            )
        )

# Розрахунок
def calculate():
    try:
        x0 = float(start_entry.get().replace(",", "."))
        b = float(end_entry.get().replace(",", "."))
        eps = float(eps_entry.get().replace(",", "."))

        if abs(x0 - 1.0) > 1e-12:
            raise ValueError("Для цієї задачі початкове значення x0 має бути 1, бо
задано y(1)=0.")

        y0 = 0.0
        history, h = euler_method(x0, y0, b, eps)
        final_x = history[-1][1]
        final_y = history[-1][2]
        result_var.set(
            f"Метод Ейлера виконано успішно.\n"
            f"Початкова умова: y(1)=0\n"
            f"Сталий крок h = {h:.6f}\n"
            f"Кількість кроків = {len(history) - 1}\n"
            f"Останнє наближення: y({final_x:.6f}) = {final_y:.6f}"
        )
        fill_table(history)
```

```
        draw_graph(history)
    except Exception as e:
        messagebox.showerror("Помилка", str(e))

# GUI
root = tk.Tk()
root.title("Лабораторна робота №6 — Метод Ейлера")
root.geometry("1100x780")
root.minsize(1000, 700)
top = ttk.Frame(root, padding=10)
top.pack(fill=tk.X)
ttk.Label(
    top,
    text="Задача Коші: y' = 3x² / (x³ + y + 1),   y(1)=0",
    font=("Arial", 12, "bold")
).grid(row=0, column=0, columnspan=6, sticky="w", pady=(0, 8))
ttk.Label(top, text="Початок відрізка x0:").grid(row=1, column=0, sticky="w")
start_entry = ttk.Entry(top, width=12)
start_entry.grid(row=1, column=1, sticky="w", padx=6)
start_entry.insert(0, "1")
ttk.Label(top, text="Кінець відрізка:").grid(row=1, column=2, sticky="w")
end_entry = ttk.Entry(top, width=12)
end_entry.grid(row=1, column=3, sticky="w", padx=6)
end_entry.insert(0, "2")
ttk.Label(top, text="Точність ε:").grid(row=1, column=4, sticky="w")
eps_entry = ttk.Entry(top, width=12)
eps_entry.grid(row=1, column=5, sticky="w", padx=6)
eps_entry.insert(0, "0.01")
btn_frame = ttk.Frame(root, padding=(10, 0, 10, 10))
btn_frame.pack(fill=tk.X)
ttk.Button(btn_frame, text="Обчислити", command=calculate).pack(side=tk.LEFT,
padx=5)
result_var = tk.StringVar()
result_label = ttk.Label(root, textvariable=result_var, justify="left",
wraplength=1000)
result_label.pack(fill=tk.X, padx=10, pady=(0, 10))
# Графік
graph_frame = ttk.LabelFrame(root, text="Графік наближеного розв’язку",
padding=10)
graph_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
fig, ax = plt.subplots(figsize=(7, 3))
canvas = FigureCanvasTkAgg(fig, master=graph_frame)
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
```

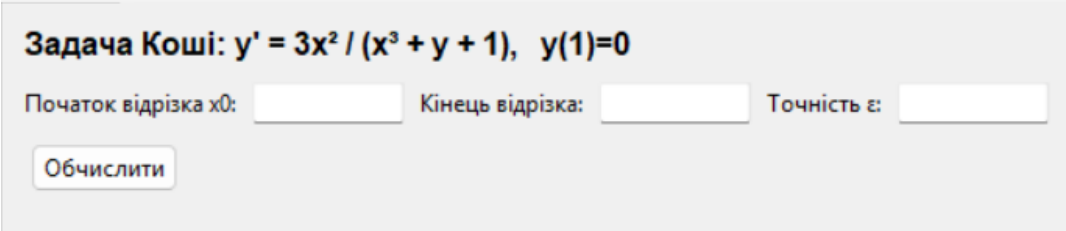
```
toolbar = NavigationToolbar2Tk(canvas, graph_frame)
toolbar.update()
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
# Таблиця
table_frame = ttk.LabelFrame(root, text="Таблиця ітерацій", padding=10)
table_frame.pack(fill=tk.BOTH, expand=False, padx=10, pady=(0, 10))
columns = ("k", "x", "y", "f(x, y)")
table = ttk.Treeview(table_frame, columns=columns, show="headings", height=8)
table.heading("k", text="k")
table.heading("x", text="x_k")
table.heading("y", text="y_k")
table.heading("f(x, y)", text="f(x_k, y_k)")
table.column("k", width=60, anchor="center")
table.column("x", width=160, anchor="center")
table.column("y", width=160, anchor="center")
table.column("f(x, y)", width=200, anchor="center")
table.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar = ttk.Scrollbar(table_frame, orient="vertical", command=table.yview)
table.configure(yscrollcommand=scrollbar.set)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

root.mainloop()
```

КІНЕЦЬ КОДУ

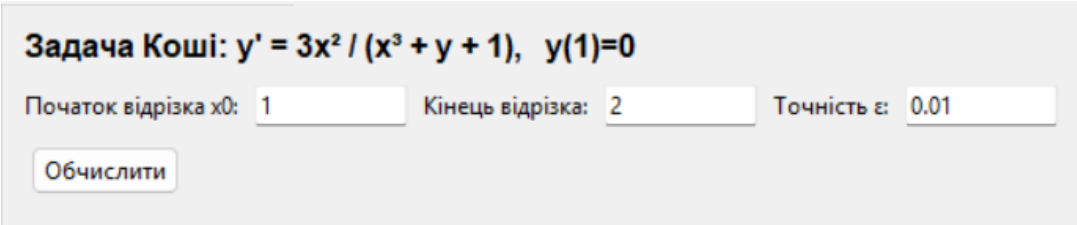
5. Результат роботи програми та графік наближеного рішення.

Дані: $y' = \frac{3x^2}{x^3+y+1}$, [1, 2] з точністю 0,01.

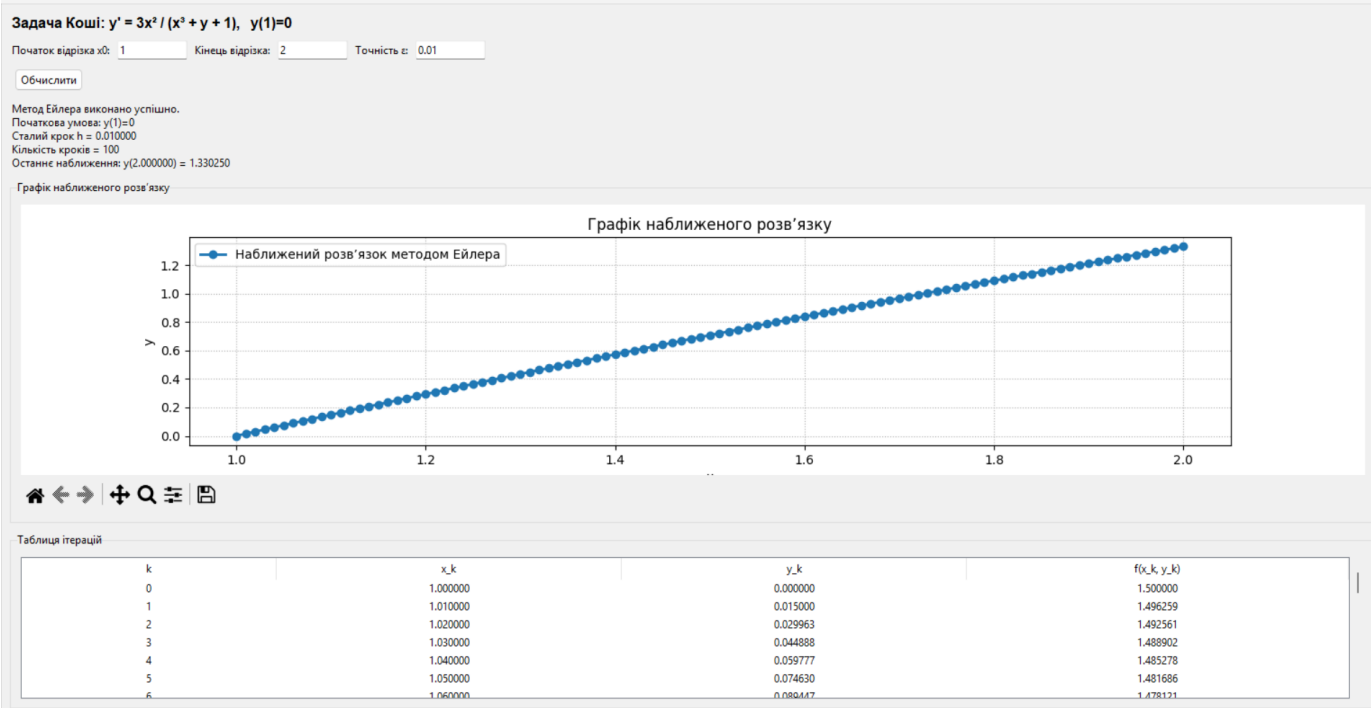
Запуск програми:



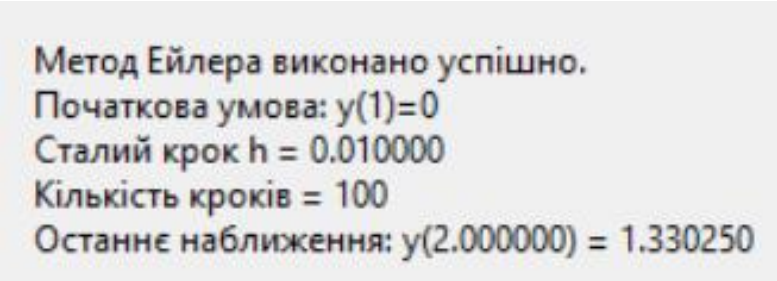
Введення даних:



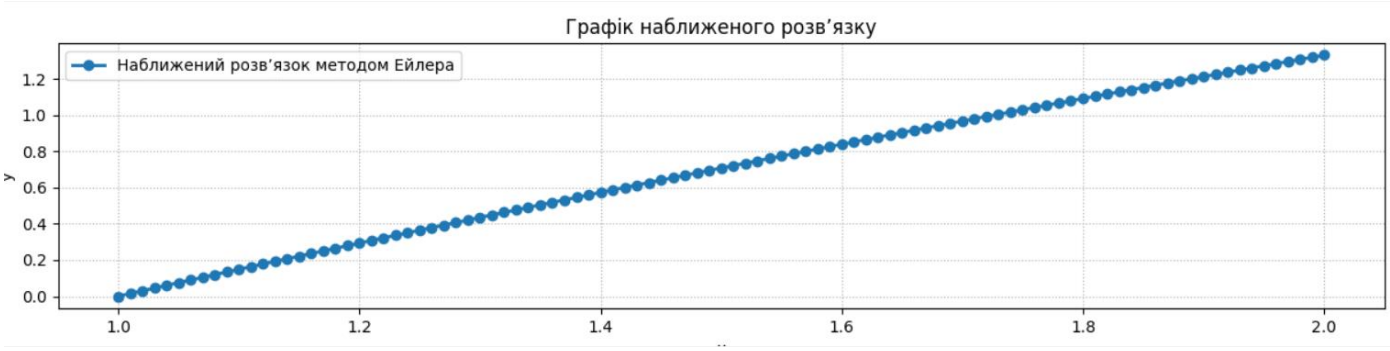
Натискання на кнопку “Обчислити”:



Перегляд результату:



Перегляд результату – Графік наближеного розв’язку:



Перегляд результату – таблиця ітерацій:

k	x_k	y_k	f(x_k, y_k)
95	1.950000	1.271688	1.177662
96	1.960000	1.283464	1.174442
97	1.970000	1.295209	1.171229
98	1.980000	1.306921	1.168024
99	1.990000	1.318601	1.164827
100	2.000000	1.330250	1.161637

6. Висновки.

У ході виконання лабораторної роботи було опановано практичні навички застосування методу Ейлера для чисельного розв’язання задачі Коші. За допомогою розробленого алгоритму було знайдено наближений розв’язок диференціального рівняння на заданому відрізку $[1, 2]$, який у кінцевій точці склав $y(2) = 1.330250$. Результати обчислень підтвердили, що метод Ейлера, попри свою простоту та перший порядок точності, є ефективним інструментом для моделювання процесів, де похибка $\varepsilon \leq 0{,}01$ є допустимою. Побудований графік наочно демонструє характер зміни функції, що відповідає теоретичним очікуванням.